JAWABAN PERTANYAAN PRAKTIKUM
MODUL III: PROTOKOL KOMUNIKASI (UART & I2C)

Ahmadin Irfan Fauzi (H1H024048)
Asisten: Athallah Tsany Satriyaji
Tanggal Percobaan: 22/04/2026
TK244005 – Praktikum Sistem Mikrokontroler | Informatika UNSOED

PERCOBAAN 3A: KOMUNIKASI SERIAL (UART)

Pertanyaan 1: *Jelaskan proses dari input keyboard hingga LED menyala/mati!*

Jawaban:

Alur lengkap dari penekanan tombol keyboard hingga LED berubah kondisi adalah sebagai berikut:

1. Pengguna mengetik karakter ('1' atau '0') pada Serial Monitor di Arduino IDE, lalu menekan tombol Enter atau klik "Send".
2. Karakter tersebut dikemas menjadi frame data UART: 1 start bit + 8 data bit + 1 stop bit, lalu dikirimkan melalui koneksi USB dari komputer ke chip USB-to-Serial (ATmega16U2) pada Arduino Uno.
3. Chip USB-to-Serial mengonversi data USB menjadi sinyal UART dan meneruskannya ke pin RX (pin 0) pada mikrokontroler ATmega328P.
4. Data masuk ke buffer penerima UART internal ATmega328P. Fungsi Serial.available() mengembalikan jumlah byte yang tersedia di buffer tersebut.
5. Kondisi if (Serial.available() > 0) terpenuhi, sehingga Serial.read() dipanggil untuk mengambil satu karakter dari buffer.
6. Program mengevaluasi nilai karakter: – Jika '1': digitalWrite(PIN_LED, HIGH) → tegangan ~5V diberikan ke pin 8 → arus mengalir melalui resistor 220Ω dan LED → LED menyala. – Jika '0': digitalWrite(PIN_LED, LOW) → tegangan 0V pada pin 8 → tidak ada arus → LED mati. – Lainnya: Serial.println("Perintah tidak dikenal") dikirim balik ke Serial Monitor.
7. Perubahan kondisi LED terjadi dalam satuan milidetik setelah karakter diterima, karena pemrosesan berlangsung sangat cepat pada frekuensi clock 16 MHz.

Pertanyaan 2: *Mengapa digunakan Serial.available() sebelum membaca data? Apa yang terjadi jika baris tersebut dihilangkan?*

Jawaban:

Serial.available() mengembalikan jumlah byte yang saat ini tersimpan di buffer penerimaan serial (maksimum 64 byte pada ATmega328P). Fungsi ini digunakan sebagai gerbang sebelum memanggil Serial.read() karena:

- Buffer serial adalah antrian (FIFO). Jika tidak ada data masuk, buffer kosong (available() = 0). Memanggil Serial.read() pada buffer kosong akan mengembalikan nilai -1, bukan karakter yang valid.
- Nilai -1 jika dimasukkan ke variabel bertipe char akan menghasilkan karakter tidak terdefinisi (nilai 0xFF atau karakter ASCII 255), sehingga kondisi if (data == '1') tidak akan pernah terpenuhi secara tepat meskipun pengguna sudah mengirim '1'.
- Tanpa pengecekan available(), fungsi loop() akan terus memanggil Serial.read() pada setiap iterasi — ribuan kali per detik — menghasilkan pemrosesan nilai -1 yang sia-sia dan membuang siklus CPU.

Jika baris Serial.available() dihilangkan, yang terjadi adalah:

- Program tetap berjalan tanpa error kompilasi maupun runtime.
- Setiap iterasi loop, Serial.read() dipanggil dan mengembalikan -1 (cast ke char menjadi 0xFF).
- Kondisi else if (data != '\n' && data != '\r') selalu terpenuhi karena 0xFF bukan \n maupun \r, sehingga pesan "Perintah tidak dikenal" akan terus-menerus tercetak ribuan kali per detik di Serial Monitor, membuat Serial Monitor banjir pesan dan tidak dapat digunakan.

Pertanyaan 3: *Modifikasi program agar LED berkedip (blink) ketika menerima input '2', dengan kondisi jika '2' aktif maka LED akan terus berkedip sampai perintah selanjutnya diberikan. Berikan penjelasan di setiap baris kode dalam bentuk README.md!*

Jawaban:

Berikut adalah kode hasil modifikasi yang mengimplementasikan mode blink non-blocking menggunakan millis():

```
#include <Arduino.h>

const int PIN_LED = 8;      // Pin digital untuk LED

// Variabel untuk mode operasi LED
char mode = '0';            // Simpan mode saat ini: '0', '1', atau '2'

// Variabel untuk blink non-blocking
unsigned long waktuSebelumnya = 0;  // Menyimpan waktu terakhir LED di-toggle
const long intervalBlink = 500;     // Interval kedip: 500 ms
bool statusLED = false;             // Status LED saat ini (nyala/mati)

void setup() {
  Serial.begin(9600);
  Serial.println("Ketik '1' nyala, '0' mati, '2' blink");
  pinMode(PIN_LED, OUTPUT);
}

void loop() {
  // ── Baca input serial jika tersedia ──
  if (Serial.available() > 0) {
    char data = Serial.read();

    if (data == '1') {
      mode = '1';
      digitalWrite(PIN_LED, HIGH);  // Nyalakan LED langsung
      Serial.println("LED ON");
    }
    else if (data == '0') {
      mode = '0';
      digitalWrite(PIN_LED, LOW);   // Matikan LED langsung
      Serial.println("LED OFF");
    }
    else if (data == '2') {
      mode = '2';                   // Aktifkan mode blink
      Serial.println("Mode BLINK aktif");
    }
```

```
      else if (data != '\n' && data != '\r') {
        Serial.println("Perintah tidak dikenal");
      }
    }

    // ── Jalankan blink jika mode == '2' (non-blocking) ──
    if (mode == '2') {
      unsigned long waktuSekarang = millis();  // Baca waktu saat ini
      if (waktuSekarang - waktuSebelumnya >= intervalBlink) {
        waktuSebelumnya = waktuSekarang;       // Simpan waktu toggle
        statusLED = !statusLED;                // Balik status LED
        digitalWrite(PIN_LED, statusLED ? HIGH : LOW);
      }
    }
  }
```

Penjelasan kode (README.md):

- const int PIN_LED = 8: mendefinisikan pin 8 sebagai konstanta untuk LED.
- char mode = '0': menyimpan mode aktif. Nilai default '0' berarti LED mati.
- unsigned long waktuSebelumnya: menyimpan millis() terakhir kali LED di-toggle. Harus unsigned long untuk menghindari overflow pada ~49 hari.
- const long intervalBlink = 500: LED berpindah kondisi setiap 500 ms sehingga periode satu kedip penuh = 1 detik.
- if (Serial.available() > 0): hanya membaca jika buffer tidak kosong, mencegah pembacaan nilai -1.
- mode = '2': flag yang memberi tahu bagian blink di bawah bahwa mode kedip aktif.
- if (mode == '2'): blok ini berjalan setiap iterasi loop. Dengan millis(), tidak ada delay sehingga pembacaan serial tetap responsif selama LED berkedip.
- waktuSekarang - waktuSebelumnya >= intervalBlink: teknik non-blocking untuk mengukur selang waktu tanpa memblokir CPU.
- statusLED = !statusLED: toggle boolean, lalu ditulis ke pin dengan operator ternary.

Pertanyaan 4: *Tentukan apakah menggunakan delay() atau millis()! Jelaskan pengaruhnya terhadap sistem.*

Jawaban:

Pilihan yang tepat adalah millis(), dengan alasan sebagai berikut:

Aspek	delay()	millis()
Cara kerja	Menghentikan seluruh eksekusi program selama N ms menggunakan busy-wait.	Membaca penghitung waktu hardware; program terus berjalan, hanya memeriksa apakah selang waktu sudah tercapai.
Penerimaan serial	TERHENTI. Karakter baru yang masuk saat delay berlangsung bisa hilang atau tertunda panjang.	TETAP AKTIF. loop() terus berjalan sehingga Serial.available() selalu diperiksa.
Responsivitas	Sistem "buta" selama delay. Latensi respons = waktu sisa delay.	Sistem responsif setiap iterasi loop (~16 µs). Latensi respons sangat kecil.

Kesimpulan	TIDAK DISARANKAN untuk sistem yang perlu baca serial sambil blink.	DIREKOMENDASIKAN. Standar industri untuk sistem embedded multi-tugas.

Jika delay(500) dipakai pada kasus ini, LED akan berkedip tetapi Arduino tidak akan bisa membaca karakter serial selama 500ms setiap setengah siklus. Akibatnya, pengguna harus menahan tombol cukup lama agar perintah berikutnya terbaca, dan perintah bisa terlewat. Dengan millis(), seluruh loop berjalan ribuan kali per detik, sehingga LED berkedip tepat waktu DAN setiap karakter baru langsung terbaca.

PERCOBAAN 3B: INTER-INTEGRATED CIRCUIT (I2C)

Pertanyaan 1: *Jelaskan bagaimana cara kerja komunikasi I2C antara Arduino dan LCD pada rangkaian tersebut!*

Jawaban:

I2C (Inter-Integrated Circuit) menggunakan dua jalur kabel bersama:

- SDA (Serial Data Line) – pin A4 Arduino: jalur dua arah untuk transfer data.
- SCL (Serial Clock Line) – pin A5 Arduino: sinyal clock yang dihasilkan master untuk sinkronisasi.

Mekanisme komunikasi Arduino → LCD berlangsung sebagai berikut:

1. Arduino (master) menarik SDA ke LOW sambil SCL tetap HIGH → ini adalah kondisi START yang menandakan awal transaksi.
2. Arduino mengirimkan 7-bit alamat slave LCD (misal 0x27 = 0100111) diikuti 1 bit arah (0 untuk tulis), total 8 bit, digeser satu per satu pada setiap pulsa SCL.
3. LCD mengenali alamatnya dan menarik SDA ke LOW sebagai ACK (Acknowledge) pada pulsa SCL ke-9.
4. Arduino mengirimkan byte data (register control LCD + nilai karakter) satu per satu. Setiap byte diikuti ACK dari LCD.
5. Setelah semua data terkirim, Arduino menghasilkan kondisi STOP (SDA naik ke HIGH saat SCL HIGH) untuk mengakhiri transaksi.
6. LCD PCF8574 (I2C expander di balik modul) menginterpretasikan byte yang diterima dan menggerakkan pin data 4-bit ke panel LCD HD44780.

Proses ini dikelola otomatis oleh library Wire.h dan LiquidCrystal_I2C, sehingga programmer hanya perlu memanggil lcd.print() tanpa mengurus protokol tingkat rendah.

Pertanyaan 2: *Apakah pin potensiometer harus seperti itu? Jelaskan yang terjadi apabila pin kiri dan pin kanan tertukar!*

Jawaban:

Konfigurasi yang benar adalah:

- Kaki kiri → GND (0V sebagai referensi bawah)
- Kaki tengah → A0 (wiper, output tegangan variabel)
- Kaki kanan → 5V (referensi atas)

Potensiometer bekerja sebagai pembagi tegangan (voltage divider). Tegangan pada wiper (kaki tengah) berbanding lurus dengan posisi putaran:

$$V_wiper = V_GND + (posisi/maksimum) \times (V_ref_atas - V_GND)$$

Jika kaki kiri dan kanan dipertukarkan (kiri ke 5V, kanan ke GND), maka arah tegangan terbalik:

- Memutar potensiometer ke kanan (yang semula menghasilkan tegangan naik) kini menghasilkan tegangan turun.
- analogRead(A0) pada posisi paling kanan menghasilkan 0 (bukan 1023), dan posisi paling kiri menghasilkan 1023 (bukan 0).
- Tampilan bar pada LCD berjalan terbalik: memutar ke kanan memperpendek bar, bukan memanjangkan.
- Ini TIDAK merusak komponen — hanya membuat perilaku berlawanan dari ekspektasi. Komponen tetap aman karena tegangan masih dalam rentang 0–5V.

Pertanyaan 3: *Modifikasi program dengan menggabungkan UART dan I2C (keduanya sebagai output) sesuai format yang ditentukan. Berikan penjelasan di setiap baris kode dalam bentuk README.md!*

Jawaban:

Berikut adalah program modifikasi yang menggabungkan output UART (Serial Monitor) dan I2C (LCD):

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Arduino.h>

// Inisialisasi LCD I2C: alamat 0x27, 16 kolom, 2 baris
LiquidCrystal_I2C lcd(0x27, 16, 2);

const int pinPot = A0;  // Pin potensiometer

void setup() {
  Serial.begin(9600);   // Inisialisasi UART dengan baud rate 9600 bps
  lcd.init();           // Inisialisasi LCD via I2C
  lcd.backlight();      // Nyalakan backlight LCD
}

void loop() {
  // 1. Baca nilai ADC (0-1023) dari potensiometer
  int nilai = analogRead(pinPot);

  // 2. Hitung tegangan: V = ADC × (5.0 / 1023)
  float volt = nilai * (5.0 / 1023.0);

  // 3. Hitung persentase: (ADC / 1023) × 100
  int persen = map(nilai, 0, 1023, 0, 100);

  // 4. Petakan ke panjang bar (0-16 karakter)
  int panjangBar = map(nilai, 0, 1023, 0, 16);

  // ── OUTPUT UART: kirim ke Serial Monitor ──────────────────────────
  // Format: "ADC: 512 Volt: 2.50 V Persen: 50%"
```

```
    Serial.print("ADC: ");
    Serial.print(nilai);
    Serial.print("\tVolt: ");
    Serial.print(volt, 2);    // 2 desimal
    Serial.print(" V\tPersen: ");
    Serial.print(persen);
    Serial.println("%");

    // ── OUTPUT I2C: tampilkan ke LCD ──────────────────────────────────────
    // Baris 1 (setCursor kol=0, baris=0): nilai ADC dan persentase
    lcd.setCursor(0, 0);
    lcd.print("ADC: ");
    lcd.print(nilai);
    lcd.print("  ");          // Hapus sisa karakter lama
    // Tambahkan persentase di posisi kolom 9
    lcd.setCursor(9, 0);
    lcd.print(persen);
    lcd.print("%   ");

    // Baris 2 (setCursor kol=0, baris=1): tampilan bar (level)
    lcd.setCursor(0, 1);
    for (int i = 0; i < 16; i++) {
      if (i < panjangBar) {
        lcd.print((char)255);  // Blok penuh (karakter ASCII 255)
      } else {
        lcd.print(" ");        // Spasi untuk bagian bar yang kosong
      }
    }

    delay(200);  // Update setiap 200ms
  }
```

Penjelasan kode (README.md):

- #include <Wire.h>: library bawaan Arduino untuk protokol I2C. Mengaktifkan fungsi komunikasi pada pin SDA (A4) dan SCL (A5).
- #include <LiquidCrystal_I2C.h>: library tambahan untuk LCD dengan modul I2C backpack (PCF8574). Menyederhanakan pengiriman data ke LCD menjadi perintah tingkat tinggi.
- LiquidCrystal_I2C lcd(0x27, 16, 2): membuat objek lcd dengan alamat I2C 0x27, 16 kolom, 2 baris.
- Serial.begin(9600): memulai komunikasi UART dengan baud rate 9600 bps. Harus dipanggil di setup() sebelum menggunakan Serial.print().
- lcd.init() + lcd.backlight(): inisialisasi LCD dan menyalakan lampu latar.
- analogRead(pinPot): membaca tegangan pada pin A0 dan mengonversinya ke nilai integer 0–1023 menggunakan ADC 10-bit internal ATmega328P.
- volt = nilai × (5.0 / 1023.0): konversi nilai ADC ke tegangan nyata. Faktor 5.0/1023 adalah resolusi per step (±4.887 mV).
- map(nilai, 0, 1023, 0, 100): memetakan nilai ADC ke rentang 0–100 secara linear untuk persentase.
- Serial.print() dengan format terstruktur: mengirimkan data ke Serial Monitor via UART. \t adalah tab untuk alignment kolom.
- lcd.setCursor(0, 0): memindahkan kursor LCD ke kolom 0 baris 0 (baris pertama).
- lcd.setCursor(0, 1): memindahkan kursor LCD ke kolom 0 baris 1 (baris kedua) untuk bar.

- (char)255: karakter blok penuh pada LCD karakter. Mencetak kotak hitam penuh sebagai elemen bar visual.
- delay(200): program berhenti 200ms sebelum pembacaan berikutnya. Cukup untuk update visual tanpa beban berlebih pada I2C bus.

Pertanyaan 4: *Lengkapi tabel berdasarkan pengamatan pada Serial Monitor (ADC, Volt, Persen).*

Jawaban:

Perhitungan berdasarkan rumus: Volt = ADC × 5.0/1023 | Persen = ADC/1023 × 100

ADC	Volt (V)	Persen (%)	Keterangan
1	0.00	0%	Hampir minimum, tegangan ≈ 0.005V, dibulatkan 0.00
21	0.10	2%	21 × 5.0/1023 = 0.1026 V ≈ 0.10 V; 21/1023×100 = 2.05% ≈ 2%
49	0.24	5%	49 × 5.0/1023 = 0.2395 V ≈ 0.24 V; 49/1023×100 = 4.79% ≈ 5%
74	0.36	7%	74 × 5.0/1023 = 0.3617 V ≈ 0.36 V; 74/1023×100 = 7.23% ≈ 7%
96	0.47	9%	96 × 5.0/1023 = 0.4692 V ≈ 0.47 V; 96/1023×100 = 9.38% ≈ 9%

Catatan: Semua nilai di atas mengacu pada tegangan referensi Vref = 5.0V dan resolusi ADC 10-bit (1023 step). Nilai aktual dapat sedikit berbeda ±0.01V karena noise ADC dan toleransi tegangan USB.

3.7 PERTANYAAN ANALISIS UMUM

Pertanyaan 1: *Sebutkan dan jelaskan keuntungan dan kerugian menggunakan UART dan I2C!*

Jawaban:

Protokol	Aspek	Keuntungan	Kerugian
UART	Kabel	Hanya butuh 2 kabel (TX, RX). Sederhana dan murah.	Hanya point-to-point (1 master – 1 slave). Tidak bisa 1-ke-banyak.
	Clock	Asinkron — tidak perlu sinyal clock bersama. Lebih fleksibel jarak jauh.	Kedua perangkat harus dikonfigurasi baud rate yang sama persis. Mismatch menyebabkan data korup.

	Kecepatan	Bisa mencapai 1–4 Mbps (high-speed UART).	Overhead per frame tinggi (start/stop bit). Efisiensi hanya ~80%.
	Penggunaan	Ideal untuk debug (Serial Monitor), GPS, Bluetooth module.	Tidak cocok untuk sistem dengan banyak perangkat serial.
I2C	Kabel	Hanya 2 kabel (SDA, SCL) untuk banyak perangkat. Hemat pin GPIO sangat signifikan.	Memerlukan resistor pull-up (biasanya 4.7kΩ) pada SDA dan SCL. Menambah komponen.
	Multi-device	Mendukung hingga 127 perangkat pada satu bus (7-bit address). Satu master bisa mengontrol banyak sensor sekaligus.	Konflik alamat jika dua perangkat berbeda memakai alamat I2C yang sama (tidak bisa diubah secara software).
	Kecepatan	Standar: 100 kbps. Fast mode: 400 kbps. Cukup untuk sensor dan display.	Lebih lambat dibanding SPI. Tidak cocok untuk transfer data besar (misal kamera).
	Penggunaan	Ideal untuk LCD, sensor suhu/kelembaban, RTC, IMU, EEPROM.	Tidak cocok untuk komunikasi jarak jauh karena rentan noise pada kapasitansi kabel tinggi.

Pertanyaan 2: *Bagaimana peran alamat I2C pada LCD (misalnya 0x27 vs 0x20)? Berikan penjelasan!*

Jawaban:

Setiap perangkat I2C wajib memiliki alamat unik 7-bit (0x00–0x7F) yang berfungsi sebagai identitas pada bus bersama. Tanpa alamat, master tidak bisa membedakan perangkat mana yang harus diajak komunikasi.

Mengapa LCD bisa memiliki alamat berbeda:

- Modul LCD I2C menggunakan chip I/O expander PCF8574 (atau PCF8574A) sebagai perantara antarmuka I2C dan bus data LCD 4-bit.
- PCF8574 memiliki 3 pin konfigurasi alamat hardware: A0, A1, A2. Setiap pin bisa HIGH (1) atau LOW (0), menghasilkan $2^3 = 8$ kombinasi alamat yang mungkin.
- PCF8574 (NXP/Texas): alamat base = 0x20 (010 0000). Dengan A0=A1=A2=1, alamat menjadi 0x27 (010 0111).
- PCF8574A (varian berbeda): alamat base = 0x38 (011 1000). Dengan A0=A1=A2=1, alamat menjadi 0x3F (011 1111).

Alamat	Chip	A2 A1 A0	Catatan	Cara mengubah
0x27	PCF8574	1 1 1	Paling umum (default pabrikan modul murah)	Solder/cabut pad A0-A2 di PCB
0x20	PCF8574	0 0 0	Semua pin alamat LOW (GND)	Hubungkan A0–A2 ke GND

0x3F	PCF8574A	1 1 1	Modul dengan chip varian A	Ganti inisialisasi: lcd(0x3F, 16, 2)

Konsekuensi alamat salah: Jika LiquidCrystal_I2C diinisialisasi dengan alamat yang tidak cocok (misalnya kode menggunakan 0x27 tetapi modul ber-alamat 0x3F), LCD tidak akan merespons sama sekali — layar tetap gelap dan tidak menampilkan apapun, meski hardware terhubung dengan benar. Solusinya: gunakan sketch I2C Scanner untuk mendeteksi alamat aktual sebelum memulai pemrograman.

Pertanyaan 3: *Jika UART dan I2C digabung dalam satu sistem (input dari Serial Monitor, output ke LCD), bagaimana alur kerja sistem tersebut? Bagaimana Arduino mengelola dua protokol sekaligus?*

Jawaban:

Arduino Uno dapat menjalankan UART dan I2C secara bersamaan karena keduanya menggunakan jalur fisik yang sepenuhnya terpisah dan memiliki hardware controller masing-masing di dalam chip ATmega328P.

Alur Kerja Sistem Terpadu:

1. Pengguna mengetik perintah di Serial Monitor → karakter dikirim via USB/UART ke pin RX Arduino.
2. Di dalam loop(), Arduino memeriksa Serial.available(). Jika ada data, Serial.read() mengambil karakter dari buffer UART.
3. Arduino memproses karakter: misalnya angka menentukan warna LED atau teks untuk ditampilkan di LCD.
4. Arduino memanggil lcd.print() → library Wire mengambil alih, mengirimkan data via jalur SDA/SCL (I2C) ke LCD.
5. LCD menampilkan teks hasil perintah. Arduino kembali ke langkah 2 untuk menunggu perintah berikutnya.

Bagaimana Arduino mengelola dua protokol sekaligus:

- **Hardware terpisah:** ATmega328P memiliki USART hardware controller untuk UART (menggunakan register UDR0, UCSR0x) DAN TWI (Two Wire Interface) hardware controller untuk I2C (menggunakan register TWDR, TWCR). Keduanya bekerja secara independen.
- **Interrupt-driven buffer:** Penerimaan UART menggunakan interrupt ISR(USART_RX_vect) yang menyimpan byte ke buffer 64-byte di RAM. CPU tidak perlu terus memantau jalur UART — hardware melakukannya secara otonom.
- **Sequential dalam software:** Meski hardware paralel, kode Arduino berjalan sekuensial. Dalam satu iterasi loop: baca UART → proses → kirim ke I2C. Karena keduanya cepat (UART buffer sudah terisi, I2C transfer ~1ms), terasa simultan bagi pengguna.
- **Tidak saling mengganggu:** Pin UART (0, 1) dan pin I2C (A4, A5) adalah pin yang berbeda secara fisik. Tidak ada kebutuhan multiplexing atau arbitrasi antara keduanya.

Keterbatasan: jika transfer I2C sangat panjang (misal banyak perangkat slave), dan UART menerima data beruntun melebihi 64 byte sebelum dibaca, terjadi buffer overflow dan data serial bisa hilang. Solusinya adalah memprioritaskan pembacaan serial atau menggunakan interrupt-based I2C.